Design e Arquitectura de Software

2025/2026

Lab. Classes - 7

The objective of this lesson will be to implement standard methods for a simple resource, field masks, and custom methods.

1 – Start by creating a Scientist Resource. This resource should include a name and area of expertise (in addition to the unique identifier – use an Integer rather than int to avoid some serialization issues). This is a static resource, it will always have these two fields (i.e. name and area of expertise cannot be removed). Implement all the standard methods for the scientist resource (GET, LIST, CREATE, UPDATE, DELETE). Use field masks as appropriate.
To implement the CREATE, UPDATE and DELETE standard methods you have to redefine the mapping to the appropriate HTPP methods. For example, in order to implement the CREATE method, the POST mapping must be redefined as illustrated here.

```
@PostMapping( value="/scientists",
              produces = MediaType.APPLICATION_JSON_VALUE))
ResponseEntity<Scientist> createScientist(@RequestBody Scientist data)
{…}
```

Notice how the method receives a parameter that is associated with the body of the HTTP request. The argument is created by deserializing the JSON in the request body (see https://www.baeldung.com/spring-request-response-body for additional details). In the case of the CREATE method, the request body should contain the JSON of the new resource to be created (minus the ID, which will be automatically created by the server). The method should return the created resource, including the ID (check the theoretical slides if you have any questions concerning the expected behavior of the CREATE standard method).

The UPDATE method requires using path variables (to identify the resource), request parameters (to identify the fields to update) and the request body (including the updated fields). Notice how, through the magic of serialization, the request body is received as a map for convenience of processing.

```
@PatchMapping(  value="/scientists/{id}",
                produces = MediaType.APPLICATION_JSON_VALUE)
ResponseEntity<Scientist> updateScientist(
           @PathVariable("id") Integer id,
           @RequestParam(value = "field", defaultValue = "*")
Set<String> fields,
           @RequestBody Map<String, Object> body)  {….}
```

- Make sure your methods return appropriate responses. For successful operations:
  - CREATE and UPDATE should return the new/modified resource, and appropriate HTTP status (201 CREATED or 200 OK, respectively)
  - CREATE should return the URI of the created resource in the Location HTTP header.

- DELETE returns 200 ok and no content.
- GET and LIST return 200 ok and the requested content.

You can use your browser to check the GET and LIST methods. Because the browser will always generate HTTP GET requests, to test other methods we must use other approach. There are multiple ways of doing it. A command line-based approach is to use the CURL command line tool to explicitly send those requests. Here is an example of usage:

```
curl --verbose -X PATCH -H "Content-Type:application/json" -d @updateData.json
http://localhost:8081/scientists/1
```

This command line will send a PATCH command to http://localhost:8081/scientist/1, with the "Content-Type:application/json" HTTP header, using the JSON data found in file updateData.json.
Here are some more examples, using an abbreviated command to POST json content directly embedded in the command line:

Creating a new scientist (CREATE)
(non-windows machines)

```
curl --json '{"name":"abel","area":"Medicine"}' http://localhost:8081/scientists
```

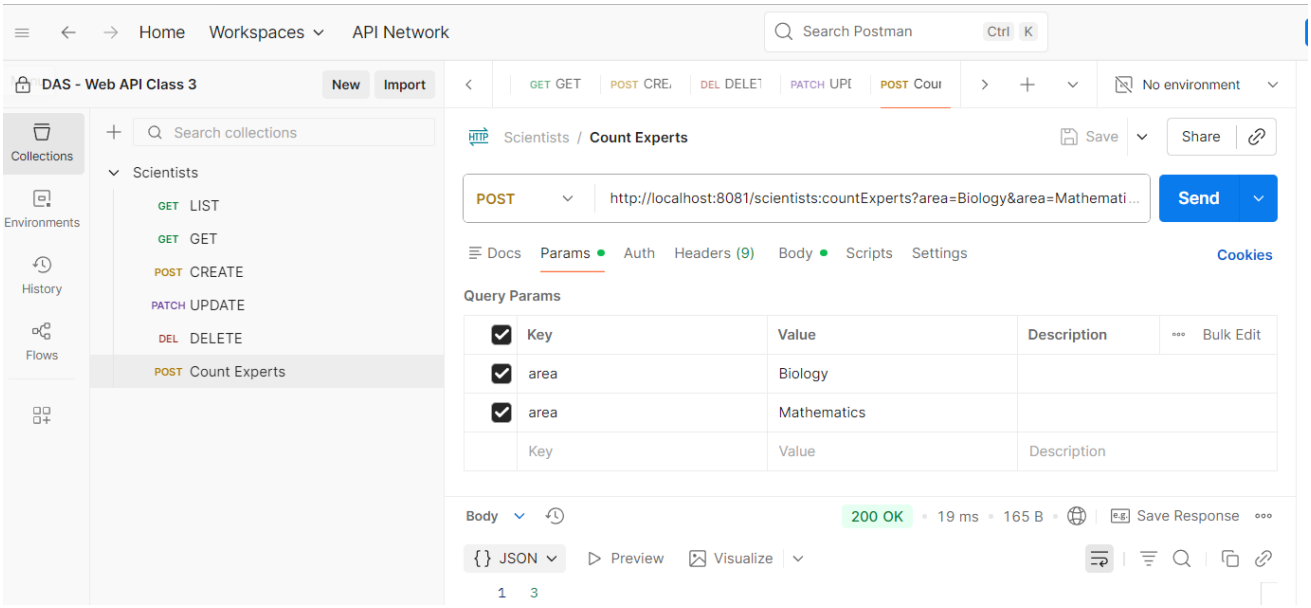
(on Windows machines you should instead do)

```
curl --json "{\"name\":\"abel\",\"area\":\"Medicine\"}" http://localhost:8081/scientists
```

Deleting a scientist (DELETE)

```
curl --verbose -X DELETE -H "Content-Type:application/json"
http://localhost:8081/scientists/1
```

Another approach is using a tool such as Postman to generate your requests. It requires you to create an account to use, but it is otherwise free. Because we want to make calls to our local machine, we will have to install locally the Postman app, available at https://www.postman.com/downloads/.
After setting up, creating an account, and logging in you will be able to create and run requests, as seen in the image bellow.



2 – Create a custom method `countExperts` that returns the number of scientists that are experts in a given area of expertise. Here is the CURL command:

Counting the number of experts in area (CUSTOM method)

```
curl --verbose -X POST -H "Accept:application/json"
http://localhost:8081/scientists:countExperts?area=Biology
```

Or counting multiple areas at once

```
curl --verbose -X POST -H "Accept:application/json"
http://localhost:8081/scientists/1:countExperts?area=Biology&area=Mathematics
```